ML 1주차 예습과제 -이륜서

Ch1. 파이썬 기반의 머신러닝과 생태계 이해

1. 머신러닝의 개념

머신러닝 : 애플리케이션을 수정하지 않고도 데이터를 기반으로 패턴을 학습하고 결과를 예측하는 알고리즘 기법을 통칭

현실 : 복잡한 조건/규칙들로 인해 일정한 패턴을 찾기 어려움, 정확도 낮음

→ 머신러닝은 데이터에서 숨겨진 패턴 인지해 해결 + 통계적 신뢰도 강화

사용 부분 : 데이터마이닝, 영상 인식, 음성 인식, 자연어 처리

구성

: 지도학습(Supervised Learning)

= [분류, 회귀, 추천 시스템, 시각/음성 감지/인지, 텍스트 분석, NLP]

+

비지도학습(Un-supervised Learning)

= [클러스터링, 차원 축소, 강화학습]

단점 : 데이터에 의존적 / Gargabe In, Garbage out

→ 데이터를 이해하고 효율적으로 가공, 처리, 추출해 최적의 데이터를 기반으로 알고리즘 구동이 중요

파이썬 : 다양한 영역에서 사용되는 개발 전문 R : 통계 전용 프로그램 언어
프로그램 언어, 직관적 문법 + 객체지향 + 함
수형 프로그래밍, 다양한 라이브러리

2. 머신러닝 주요 패키지

- 머신러닝 패키지
  - 사이킷런(Scikit-Learn) : 데이터 마이닝 기반
- 행렬/선형대수/통계 패키지

  : 머신러닝의 이론적 배경은 선형대수와 통계
  - 넘파이(Numpy) : 선형대수를 다루는 패키지 (행렬기반 데이터)
  - 사이파이(SciPy) : 자연과학과 통계를 위한 패키지
- 데이터 핸들링
  - 판다스 : 파이썬 세계의 대표적 데이터 처리 패키지 / 2차원 데이터 처리 / 넘파이보다 편리하게 데이터 처리
  - 맷플롯립(Matplotlib)을 호출해 시각화 기능 지원
- 시각화
  - 맷플롯립 : 파이썬의 대표적인 시각화 패키지 / 세분화된 API로 익히기 번거롭다

    → 시본(Seaborn) : 맷플롯립 기반 / 판다스와의 쉬운 연동, 함축적 API, 분석을 위한 다양한 유형의 그래프&차트 제공

    세밀한 제어는 여전히 맷플롯립 기반
- 아이파이썬(IPython, Interactive Python)툴

  : 대화형 파이썬 툴, 프로그래밍 + 설명적인 요소, 특정 코드 영역별로 개별 지원 → 코드 이해 명확
  - 주피터 노트북(Jupyter Notebook)

3. 넘파이(Numpy, Numerical Python)

- 선형대수 기반의 프로그램을 쉽게 만들도록 지원하는 패키지
- 루프 사용하지 않고 대량 데이터의 배열 연산 가능 → 빠른 배열 연산 속도
- C / C++ 같은 저수준 언어 기반의 호환 API 제공
- 다양한 데이터 핸들링 기능 제공

1) 넘파이 ndarray 개요

```
import numpy as np
```

- ndarray : 넘파이의 기반 데이터 타입 → 다차원(Multi - dimension)배열을 쉽게 생성하고 다양한 연산 수행
- array( ) 함수 : 파이썬의 리스트와 같은 다양한 인자를 입력 받아 ndarray로 변환 ← np.array()로 사용
    - shape 변수 : ndarray의 크기(=행과 열의 수)를 튜플 형태로 지님 ← ndarray.shape

배열	shape	데이터수
[1,2,3]	(3,)	1차원, 3개 데이터
[[1,2,3], [2,3,4]]]]	(2,3)	2차원, 2*3 = 6개 데이터
[[1,2,3]]	(1,3) - 1개 row, 3개 col	2차원, 3개 데이터

ndaary.ndim : array의 차원을 알려줌

리스트 [ ] : 1차원, [[ ]] : 2]]

2)ndarray의 데이터 타입

- 숫자 값, 문자열 값, 불 값 등 모두 가능
    - 숫자형 : int형(8bit, 16bit, 32bit), unsigned int형(8bit, 16bit, 32bit), float형(16bit, 32bit, 128bit)
    - 한 개의 ndarray의 객체에 int와 float 공존 X
- 한 객체 안에는 같은 유형의 데이터만 가능
    - 만약 다른 데이터 유형이 섞인 경우 : 더 큰 데이터 타입으로 형 변환

  ex) int & 문자열 → 문자열, int & float → float
- dtype 속성 : ndarray내의 데이터 타입 확인 → array1.dtype
- astype( )메서드 : 데이터값의 타입 변경, 메모리를 더 절약할 때 사용

ex) array_int = np.array([1,2,3])

array_float = array_int.astype('float 64') → int를 float 형으로 변경

3) ndarray를 편리하게 생성하기

arange( )	range( )와 유사	np.array(10) : 0부터 9까지의 연

	→ 0부터 함수 인자 값 -1까지	속된 숫자
zeros( )	인자로 튜플 형태의 shape값 입력 → 모든 값을 0으로	zero_array = np.zeros((3,2), dtype = 'int32')
ones( )	인자로 튜플 형태의 shape 값 입력 → 모든 값을 1로	one_array = np.ones((3,2))

zeros( )와 ones( )에서 → 함수 인자로 dtype을 정해주지 않으면 float 64형의 데이터로 됨

4) ndarray의 차원과 크기를 변경하는 reshape( )

reshape( )함수	ndarray를 특정 차원 및 크기로 변환 reshape(로우, 칼럼) → 지정된 사이즈로 변경 불가능하면 오류 발생 -1을 인자 : 나머지 인자 기준으로 자동 변환 reshape(-1,1) 자주 사용 : 여러 개의 로우를 가지되 반드시 1개 칼럼을 가진 ndarray로 변환 보장	array1 = np.arange(10) array2 = array1.reshape(2,5) • reshape(4,3) : 오류 발생
tolist( )메서드	ndarray를 리스트 자료형으로 변환	array3d.shape( )

5) 넘파이의 ndarray의 데이터 세트 선택하기 - 인덱싱(Indexing)

- 특정한 데이터만 추출 : 원하는 위치의 인덱스 값을 지정하면 해당 위치 데이터 반환

  1개 데이터 추출 : 인덱스 값을 [ ]안에 입력

  2차원 경우 : 콤마(,)로 분리된 로우와 칼럼 위치의 인덱스로 접근

  - axis0 : 로우(=행) 방향의 축, axis1 : 칼럼(=열) 방향의 축, axis2 : 높이
- 슬라이싱(Slicing) : 연속된 인덱스상의 ndarray를 추출, 시작 인덱스 : 종료 인덱스 → 시작 ~ 종료 인덱스 -1까지의 데이터 반환
- 팬시 인덱싱(Fancy Indexing) : 일정한 인덱싱 집합을 리스트 또는 ndarray 형태로 지정해 해당 위치에 있는 데이터의 ndarray 반환
- 불린 인덱싱(Boolean Indexing) : True / False 값 인덱싱 기반으로 True에 해당하는 인덱스 위치에 있는 ndarray 반환

6) 행렬의 정렬 - sort( )와 argsort( )

(1)행렬 정렬

np.sort( ) : 넘파이에서 sort( )를 호출, 원 행렬 그대로 유지한 채 원 행렬의 정렬된 행렬 반환

ndarray.sort( ) : 행렬 자체에서 sort( )를 호출, 원 행렬 자체를 정렬한 형태로 변환하며 반환 값은 None

- 기본은 오름차순 / 내림차순 정렬 : [::-1] 적용
- 행렬 2차원 이상 : axis = 0 → 로우방향 정렬, axis = 1 → 칼럼방향 정렬

(2)정렬된 행렬의 인덱스를 반환하기

np.argsort( ) : 정렬 행렬의 원본 행렬 인덱스를 ndarray 형으로 반환

→ 원본 행렬의 정렬 시 행렬 인덱스 값 구함

argsort( ) : 넘파이에서 활용도 높다, 실제 값과 그 값이 뜻하는 메타 데이터를 별도의 ndarray로 가져야

7) 선형대수 연산 - 행렬 내적과 전치 행렬 구하

(1) 행렬 내적(행렬 곱)

- np.dot(A,B) 이용
- 행렬 A의 행 = 행렬 B의 열

(2) 전치 행렬

- 원 행렬에서 행과 열의 위치를 교환한 원소로 구성
- 원 행렬 : A, 전치 행렬 : A^T
- np.tranpose(A) 이용

4. 데이터 핸들링 - 판다스

- 행과 열로 이뤄진 2차원 데이터를 효율적으로 가공 / 처리
- DateFrame : 판다스의 핵심 객체 / 2차원 데이터를 담는 데이터 구조체 / 칼럼이 여러 개인 데이터 구조체 / 여러 개의 Series로 구성
- Index : 개별 데이터를 고유하게 식별하는 Key 값 → Series와 DateFrame 모두 Index를 Key값으로 가짐
    - Series : 칼럼이 하나 뿐인 데이터 구조체

1) 판다스의 시작 - 파일을 DataFrame으로 로딩, 기본 API

- 다양한 포맷으로 된 파일을 DataFrame으로 로딩할 수 있는 편리한 API 제공

read_csv(filepath_or_buffer, sep=',')	CSV(칼럼을 ,로 구분) 파일 포맷 변환을 위한 API sep 인자로 구분, filepath : 가장 중요한 인자
read_table( )	필드 구분 문자가 \t, 나머지 기능은 read_csv와 동일
read_fwf( )	Fixed Width 고정 길이 기반의 칼럼 포맷을 DateFrame으로 로딩하기 위한 API

- DateFrame.head( ) : DateFrame의 맨 앞에 있는 N개의 로우 반환

→ head(3) : 맨 앞 3개의 로우 반

- info( ) 메서드 : 총 데이터 건수와 데이터 타입, Null 건수
- describe( ) 메서드 : 칼럼별 숫자형 데이터값의 n-percentile 분포도, 평균값, 최댓값, 최솟값

  → 숫자형(int, float) 칼럼의 분포도만 조사 (object 타입은 출력에서 제외)
- value_counts( ) 메서드 : 해당 칼럼값의 유형과 건수 확인, 지정된 칼럼의 데이터값 건수 반환, 데이터 분포도 확인에 유
- DateFrame['칼럼명'] : 해당 칼럼에 해당하는 Series 객체 반

2) DateFrame과 리스트, 딕셔너리, 넘파이 ndarray 상호 변환

사이킷런의 많은 API는 DateFrame을 인자로 입력받을 수 있지만, 기본적으로 넘파이 ndarray를 입력 인자로 사용 → DateFrame과 넘파이 ndarray 상호 간의 변환 빈번하게 발생

(1)넘파이 ndarray, 리스트, 딕셔너리 → DateFrame로 변환

DateFrame : 칼럼명 O [넘파이 ndarray, 리스트는 X] → 데이터 핸들링 가능

- DateFrame으로 변환 시 칼럼명 지정 (지정X → 자동 할당)

data	리스트, 딕셔너리, 넘파이 ndarray를 입력 받는 생성인자
columns	칼럼명 리스트를 입력받는 생성인ㄴ자

딕셔너리 : 키(Key)는 칼럼명, 값(Value)는 키에 해당하는 칼럼 데이터로 변환

→ 키 : 문자열, 값 : 리스트(또는 ndarray)형태로 딕셔너리 구성

(2)DateFrame → 넘파이 ndarray, 리스트, 딕셔너리로 변환

넘파이 ndarray로 변환	DataFrame 객체의 values 이용	array = df_dict.values
리스트로 변환	values로 얻은 ndarray에 tolist( )호출	list = df_dict_values.tolist()
딕셔너리로 변환	DataFrame 객체의 to_dict( )메서드 호출 → 인자로 'list' 입력 시 딕셔너리 값이 리스트형으로 반환	dict = df_dict.to_dict('list')

3) DataFrame의 칼럼 데이터 세트 생성과 수정

- 칼럼 추가 : DataFrame [ ]내에 새로운 칼럼명 입력 후 값 할당
- 기존 칼럼 Series의 데이터로 새로운 칼럼 Series 생성, 모든 데이터 세트에 일괄적 적용
- 기존 칼럼 값도 일괄 업데이트 → 업데이트 원하는 칼럼 Series를 DateFrame[ ]내에 칼럼명으로 입력 후 값을 할당

4) DataFrame 데이터 삭제

- drop( ) 메서드 이용

→ DataFrame.drop(labels = None, axis = 0, index = None, columns = None, level = None, inplace = False, errors = 'raise')

labels에 원하는 칼럼명 입력

- axis = 0 : 로우 축 방향으로 드롭 → 특정 로우를 드랍, 이상치 데이터 삭제
- axis = 1 : 칼럼 축 방향으로 드롭 → 특정 칼럼을 드랍, 많은 경우에서 사용

axis	• 0입력 : 로우 축 방향으로 드롭 → 특정 로우 드랍, 이상치 데이터 삭제

	• 1입력 : 칼럼 축 방향으로 드롭 → 특정 칼럼 드랍, 많은 경우에서 사용
Inplace	• inplace = False : 자기 자신의 DataFrame은 삭제 X, 삭제된 결과 DataFrame을 반환 • inplace = True : 자신의 DataFrame의 데이터 삭제 → 반환값 : None

5) Index 객체

정의 : DataFrame, Series의 레코드를 고유하게 식별하는 객체

- DataFrame.index 또는 Series.index 속성으로 객체 추출
- 식별성 데이터를 1차원 array로 지님
- ndarray와 유사하게 단일 값 반환 및 슬라이싱 가능
    - 한 번 만들어진 DataFrame 및 Series의 Index 객체는 함부로 변경X
- Series 객체에 연산 함수 적용 시 Index는 연산에서 제외, 오직 식별용으로만
- DataFrame 및 Series에 reset_index( )메서드 수행하면 새롭게 인덱스를 연속 숫자형으로 할당 + 기존 인덱스는 'index'라는 새로운 칼럼명으로 추가

6) 데이터 셀렉션 및 필터링

(1) DataFrame vs 넘파이 [ ] 연산자

- 넘파이 [ ] 연산자 : 행의 위치, 열의 위치, 슬라이싱 범위 등을 지정
- DataFrame [ ] 연산자 : [ ]는 칼럼명을 지정해 칼럼 지정 연산에 사용 또는 불린 인덱스 용도로 사용

(2)DataFrame iloc[ ] 연산자

- iloc[행 위치 정숫값, 열 위치 정숫값] : 위치 기반 인덱싱, 행과 열 위치를, 0을 출발점으로 하는 세로축, 가로축 좌표 정숫값으로 지정하는 방식
- iloc[ ]에 DataFrame의 인덱스 값이나 칼럼명 입력 → 오류
    - -1 : 마지막 데이터 값
    - 슬라이싱과 팬시 인덱싱 제공 but 불린 인덱싱 제공 X

(3)DataFrame loc[ ] 연산자

- loc[인덱스값, 칼럼명] : 명칭 기반 인덱싱, 데이터 프레임의 인덱스 값으로 행 위치를 칼럼의 명칭으로 열 위치를 지정

- 시작형 : 종료점 = 종료점도 포함한 위치에 있는 데이터도 반환 → 명칭은 숫자형이 아닐 수 있어서

7) 불린 인덱싱

객체가 DataFrame이므로 원하는 칼럼명만 별도로 추출

- and = &, or = |, Not = ~

8) 정렬, Aggregation 함수, GroupBy 적용

(1) sort_values( )

- DataFrame, Series의 정렬
  - by : 특정 칼럼으 입력하면 해당 칼럼으로 정렬 수행
  - ascending = True : 오름차순, ascending = False : 내림차순 [기본은 오름차순]
  - inplace = False : sort_values( ) 를 호출한 DataFrame은 그대로 유지, 정렬된 DataFrame을 결과로 반환 [기본 형태]
  - inplace = True : 호출한 DataFrame의 정렬 결과 그대로 적용

(2) Aggregation 함수 적용

- min( ), max( ), sum( ), count( ) 등이 존재
- 바로 aggregation을 호출할 경우 모든 칼럼에 해당 aggregation을 적용
- 특정 칼럼에 적용하기 위해서는 DataFrame에 대상 칼럼만 추출해 적용

(3) groupby( ) 적용

- groupby( ) 사용 시 입력 파라미터 by에 칼럼 입력 → 대상 칼럼으로 groupby 된다
- groupby( )를 호출하면 DataFrameGroupBy라는 또 다른 형태의 DataFrame 반환
- DataFrame에 groupby( )를 호출해 반환된 결과에 aggregation함수를 호출 → groupby( )대상 칼럼을 제외한 모든 칼럼에 해당 aggregation 함수를 적용
- 특정 칼럼만 aggregation 함수 적용하려면 groupby( )로 반환된 DataFrameGroupBy 객체에 해당 칼럼을 필터링 한 뒤 aggregation 함수 적용
- 서로 다른 aggregation 함수 적용 : 여러 개의 aggregation 함수명을 DataRameGroupBy 객체의 agg( )내에 인자로 입력해 사용

9)결손 데이터 처리하기

- 판다스는 결손 데이터를 처리하는 편리한 API 제공
- 결손 데이터 = 칼럼에 값이 없는 NULL의 경우 = 넘파이 NaN
    - NaN 값은 함수 연산 시 제외

isna( )	NaN 여부 확인 → True / False sum( )함수 : 결손 데이터 개수 구하기(True : 1, False : 0)
fillna( )	NaN 값을 다른 값으로 대체 fillna( )를 이용해 반환 값 다시 받거나 inplace = True를 추가해야 데이터 세트 값 변경

10)apply lambda 식으로 데이터 가공

- 판다스는 apply 함수에 lambda 식을 결합해 DataFrame이나 Series 레코드별로 데이터를 가공하는 기능 제공
- lambda 식 : 파이썬에서 함수형 프로그래밍 지원을 위해 생성 → 함수의 선언과 함수 내의 처리를 한 줄의 식으로 쉽게 변환
    - 여러 개의 값을 입력 인자로 사용 : map( )함수 사용
    - if else 지원, if 절의 경우 if 식보다 반환 값을 먼저 기술